ORACLE

Implementing an Abstraction for Verifiable Credentials and Zero Knowledge Proofs

Internet Identity Workshop, October 2024

Mark Moir (mark.moir@oracle.com)

Architect

Oracle Labs

Joint work with Harold Carr (harold.carr@oracle.com)

About us

- Full-time
  - Harold Carr (https://www.linkedin.com/in/haroldcarr)
    - Blockchain fault tolerance and scalability, Infiniband transport, distributed systems, logic circuit simulation, …
  - Mark Moir (https://www.linkedin.com/in/markmoir)
    - Blockchain fault tolerance and scalability, synchronization primitives, concurrent algorithms, formal verification, …

- Former interns
  - Christoph Braun, Ph.D. student, Karlsruhe Institute of Technology
    - Contributed to use case demo and machinery to enable it (Summer, 2023)
  - Henry Blanchette, Ph.D. student, University of Maryland, College Park
    - Contributed to Haskell prototype and translating it to Rust (Summer, 2024)

Agenda

1. “Elevator pitch” and research overview
2. Background and example use case
3. Motivation for and overview of our abstraction
4. Status: implementation and testing framework
5. Wrapup

Agenda

1. “Elevator pitch” and research overview
2. Background and example use case
3. Motivation for and overview of our abstraction
4. Status: implementation and testing framework
5. Wrapup

“Elevator Pitch”: Using Verifiable Credentials and Zero Knowledge Proofs to balance privacy and accountability

- Collecting too much information creates liability, compliance and privacy issues
- Collecting too little precludes accountability
- Verifiable Credentials and Zero Knowledge Proofs can help to strike an effective balance
- But, many VC projects and standards don’t enable such use of ZKPs and/or are tied too tightly to specific cryptography libraries, limiting choice and progress

Icon attributions on last slide: 1, 2

Overview of our research

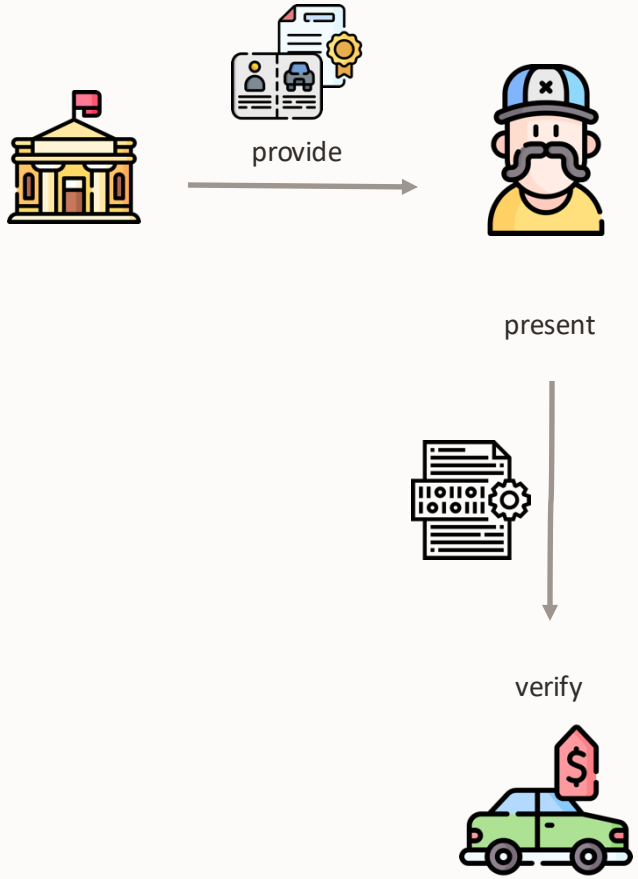
- Learning about these technologies, projects, standards efforts
- Demonstrating (internally) potential of technologies
- Developing an experimental abstraction to decouple VCs from underlying cryptography
- Implementing and testing our abstraction, instantiated with multiple cryptography libraries
- Seeking feedback and engagement internally and externally

Agenda

1. “Elevator pitch” and research overview
2. Background and example use case
3. Motivation for and overview of our abstraction
4. Status: implementation and testing framework
5. Wrapup

Verifiable Credentials: basic roles

Issuer issues credential that includes attributes (e.g., about Holder)



Holder presents “something” to Verifier

Verifier verifies the presented “something”

See last slide for icon attributions

Strawman approach: Use traditional digital signatures

- Issuer signs message that includes attributes
- Holder presents message and signature to Verifier
- Verifier verifies signature

- Privacy implications
  - Holder must reveal entire message to enable verification
  - Holder presents same signature every time, enabling correlation, tracking, etc.
- Regulations, best practices require compliance with Data Minimization principle:
  - Collect only necessary information

Using Zero Knowledge Proofs

- Issuer signs message that includes attributes
- Holder presents ~~message and signature~~ proof of knowledge of Issuer's signature
- Verifier verifies ~~signature~~ proof

- Zero Knowledge Proofs
  - Prove knowledge of something (e.g., Issuer's signature) *without* disclosing it
  - Different proof each time; prevents unwanted correlation
  - Reveal selected attributes, hide others
  - Prove properties about something without disclosing it, e.g.:
    - Predicates such as DOB > 20 years ago
    - Set (non)membership,
    - Many others

Zero Knowledge Proofs

	Zero-knowledge Proofs:	est. ca.
Privacy	• *Proof of Knowledge of Signature (selective disclosure)*	2004
	• *Range Proofs (range membership)*	2008
	• *Cryptographic Accumulators (set membership)*	2008
Accountability	• *Verifiable Encryption (encrypted disclosure)*	1998

Zero Knowledge Proofs

Commit-and-prove technology enables composing different ZKPs without compromising ZK

LegoSNARK: Modular Design and Composition of Succinct Zero-Knowledge Proofs

Matteo Campanelli[1], Dario Fiore[1], and Anaïs Querol[1,2]

[1] IMDEA Software Institute
[2] Universidad Politécnica de Madrid

matteo.campanelli@imdea.org
dario.fiore@imdea.org
anais.querol@imdea.org

Full Version

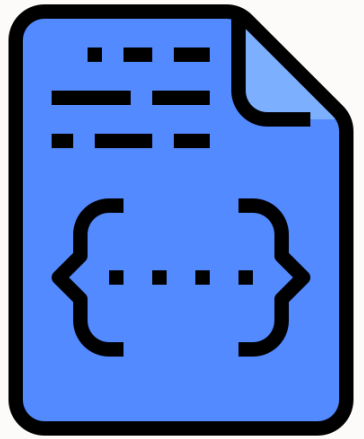
Abstract. We study the problem of building non-interactive proof systems modularly by linking small specialized "gadget" SNARKs in a lightweight manner. Our motivation is both theoretical and practical. On the theoretical side, modular SNARK designs would be flexible and reusable. In practice, specialized SNARKs have the potential to be more efficient than general-purpose schemes, on which most existing works have focused. If a computation naturally presents different "components" (e.g. one arithmetic circuit and one boolean circuit), a general-purpose scheme would homogenize them to a single representation with a subsequent cost in performance. Through a modular approach one could instead exploit the nuances of a computation and choose the best gadget for each component.
Our contribution is LegoSNARK, a "toolbox" (or framework) for commit-and-prove zkSNARKs (CP-SNARKs) that includes:

IACR Cryptol. ePrint Arch. 2019

	Zero-knowledge Proofs:	est. ca.
Privacy	• *Proof of Knowledge of Signature (selective disclosure)*	2004
Privacy	• *Range Proofs (range membership)*	2008
Privacy	• *Cryptographic Accumulators (set membership)*	2008
Accountability	• *Verifiable Encryption (encrypted disclosure)*	1998
	• ***and composites of those!***	**2016/2019 (in theory!)**

Implementation based on Commit-and-Prove, 2021-2024

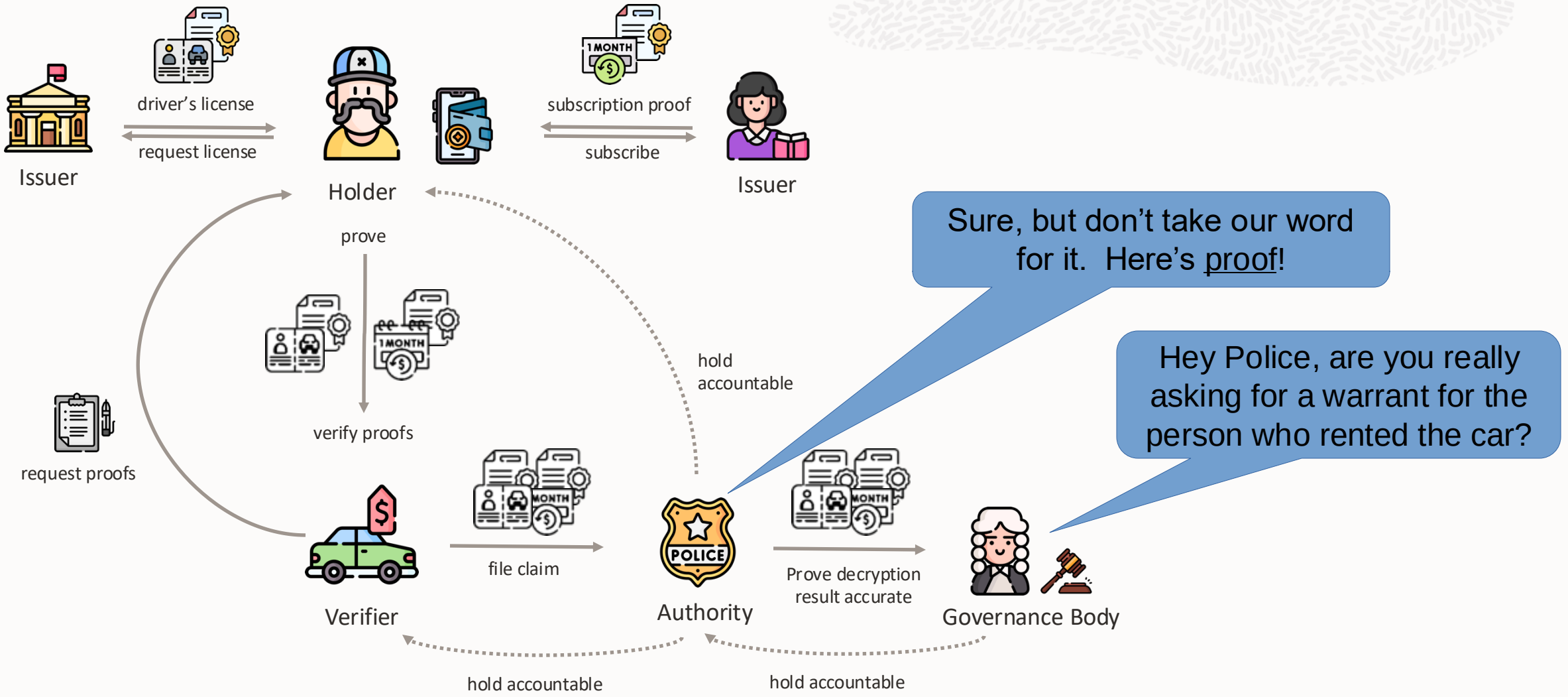
- Open source DockNetwork crypto library uses commit-and-prove to implement “proof system” that enables composing any/all of:
  - Efficient BBS+ signatures, with Selective Disclosure
  - Range proofs (e.g., DOB > 20 years ago)
  - Cryptographic accumulators (supporting privacy-preserving revocation, for example)
  - Other ZKPs (any zk-SNARK that can be expressed in R1CS, e.g., created using Circom)
  - Verifiable encryption
  - More (secret sharing, distributed key generation, …)



DockNetwork crypto (DNC) library

See last slide for icon attributions (3)

Example use case - Accountability in privacy-preserving authentication for services



Icons by Freepik from flaticon.com

Accountability variations

- As described so far, Authority and Verifier could theoretically collude to identify every renter, even well-behaved ones
- Could reverse roles, so Authority (e.g. Police) must request Governance Body (e.g. Court) to identify renter
- DockNetwork crypto library also supports secret sharing (though our work does not yet address it)
  - Could require *both* Authority *and* Governance Body to cooperate with Verifier
  - Could require *k* of *t* parties to identify renter

Agenda

1. “Elevator pitch” and research overview
2. Background and example use case
3. Motivation for and overview of our abstraction
4. Status: implementation and testing framework
5. Wrapup

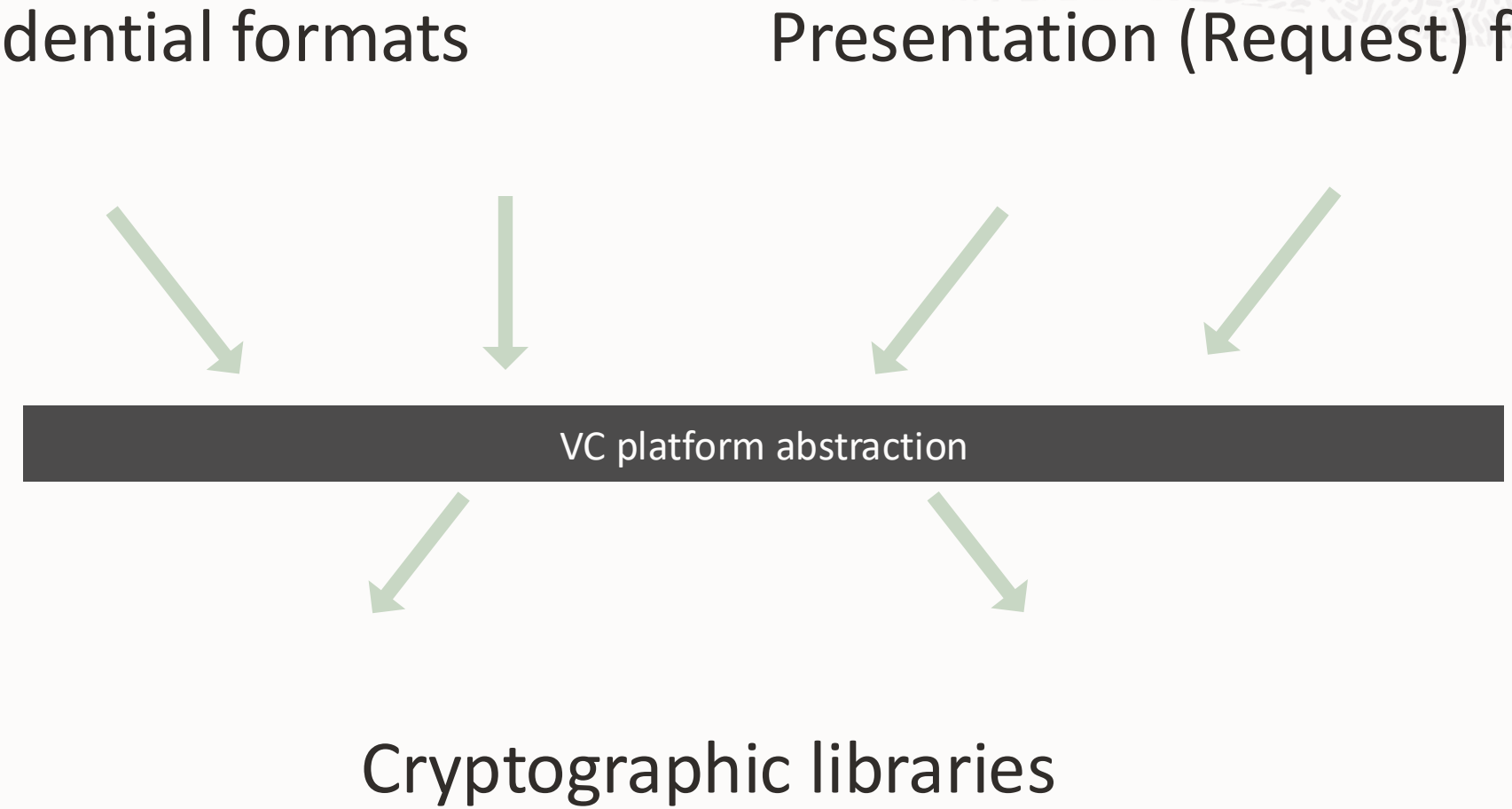
Motivation for abstraction

- Facilitate providing strong privacy and accountability features for any Credential and Presentation (Request) formats
- Facilitate adoption of new and/or improved cryptography libraries by any Credential and Presentation (Request) formats
- Enable evolution of Credential and/or Presentation (Request) formats and/or cryptography libraries, relatively independently of each other
- Facilitate experimentation with and evaluation of different credential formats, libraries, etc.
- Reduce risk for projects and products that aim to effectively balance privacy and accountability
- …

Some potential benefits

- Enable people (not just developers with expertise to use specialized cryptography libraries) to more easily express/understand/analyze/audit use case requirements in different ways
- Enable reports/summaries/warnings for use case requirements in various formats, levels of detail
  - Example: User requests to disclose a verifiably encrypted attribute: likely a mistake. Conceivably intentional, so we allow it, but generate a warning
  - Similarly for disclosing accumulator members (thus leaking correlatable info)
- Facilitate switching underlying cryptography libraries for various reasons:
  - Performance (comparison), features, stronger security guarantees (e.g., post-quantum), more rigorous proofs, more favourable license, health of project, …
- Could enable formal proofs of privacy properties about use cases, based on assumptions about guarantees made by underlying cryptography (stated formally, perhaps proved formally)
- Enable extensive and extendible test suites, independent of underlying library and implementation language

An abstraction for decoupling credential formats from underlying cryptographic libraries

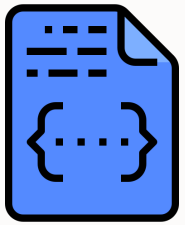


An abstraction for decoupling credential formats from underlying cryptographic libraries

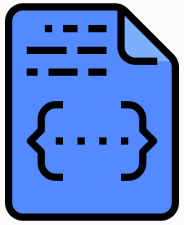
W3C

VC platform abstraction

Hyperledger Ursa



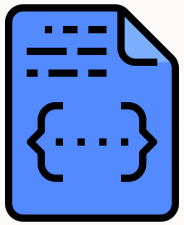
Hyperledger AnonCreds



Hyperledger AnonCreds 2.0

Post-quantum?

DockNetwork crypto



See last slide for icon attributions (3,4)

Partial list of things we think should live above the abstraction

- VC format (W3C VCDM, AnonCreds v*, ACDC,…)
- Attribute names (if any), but recall “internal debate”
- Negotiation of presentation requirements
- Communication protocols
- VC contents
  - Example: questions such as whether credentials require metadata could/should be be separate from underlying cryptography
  - Similarly for link secrets
  - Similarly for accumulators (e.g., zero, one, or multiple revocation managers?)
- Rules, policies, roles
  - Example: AnonCreds v2 assumes Authority = Issuer = Revocation Manager
    - We don’t think this should be assumed (e.g., Authority could be Police, some other government entity, some legal entity, multiple entities, etc., determined by use case)
    - Regardless, such decisions should be separate from underlying cryptography
- …

Actually, two related abstractions

- “Crypto Interface”
  - Defines minimal requirements for underlying cryptography library to implement
- “Platform API”
  - Provides access to all cryptographic functionality via higher-level abstraction, provides various validation, warnings, etc., while remaining independent of underlying library and Credential and Proof Request formats

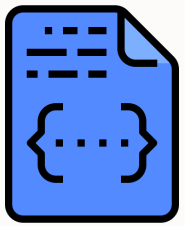
Separate minimal library requirements from higher-level functionality

W3C

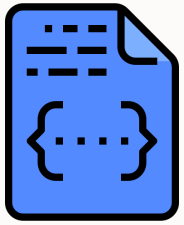
Platform API

Crypto Interface

Hyperledger Ursa



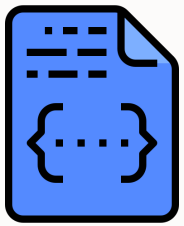
Hyperledger AnonCreds



Hyperledger AnonCreds 2.0

Post-quantum?

DockNetwork crypto



See last slide for icon attributions (3,4)

Separate minimal library requirements from higher-level functionality

AC/DC

Hyperledger ANONCREDS

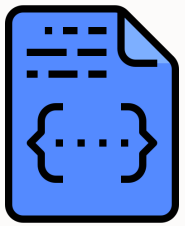
W3C

Platform API

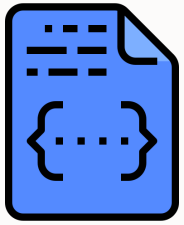
Crypto Interface

"General"

Hyperledger Ursa



Hyperledger AnonCreds

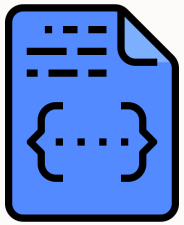


Hyperledger AnonCreds 2.0

...

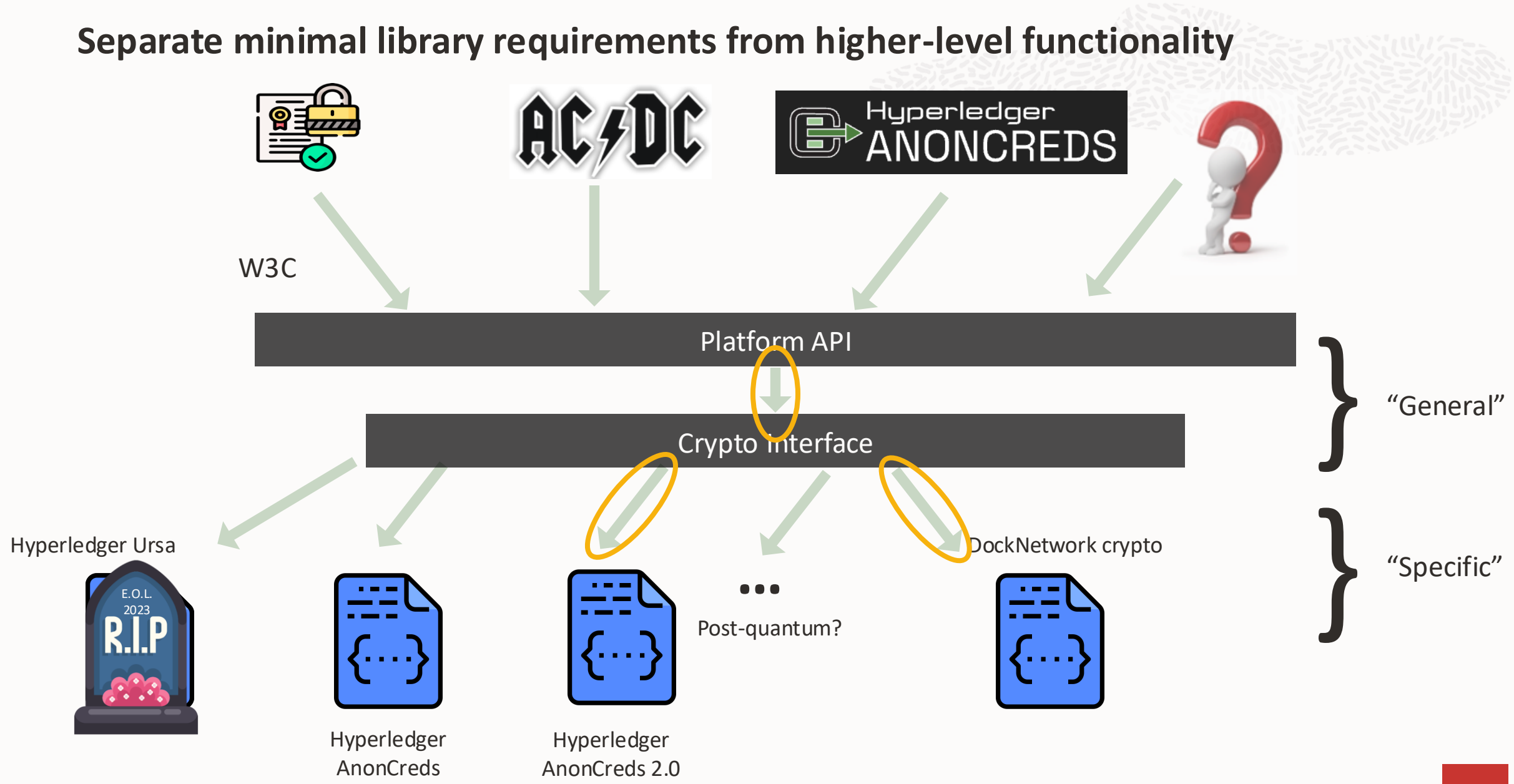
Post-quantum?

DockNetwork crypto



See last slide for icon attributions (3,4)

Separate minimal library requirements from higher-level functionality



See last slide for icon attributions (3,4)

Abstracting types via “OpaqueMaterial”

```
-- SignerData has internal structure, exposed by the abstraction

data SignerData = SignerData
  { signerPublicData :: SignerPublicData
  , signerSecretData :: SignerSecretData
  }

-- Components need only be passed back to interface (e.g., to sign or verify
-- a signature); hides library-specific implementation detail, ensuring same
-- interface can be implemented by multiple cryptography libraries

data SignerPublicData = SignerPublicData OpaqueMaterial
data SignerSecretData = SignerSecretData OpaqueMaterial
```

The abstraction in more detail (1/2)

- Abstraction is agnostic to credential format (e.g., W3C, AnonCreds v*, …)
- Various “opaque types” that user receives from interface and sends back (perhaps indirectly), with examples of included information when implemented over DockNetwork crypto library:
  - **`SignerPublicData`**
    - Public key, signature parameters
  - **`SignerSecretData`**
    - Secret key
  - **`DataForVerifier`**
    - Proof, values of revealed attributes
  - **`AuthorityPublicData`**
    - Chunk size, commitment generators, encryption generators, encryption key, Groth16ProvingKey, …
  - **`AuthoritySecretData`**
    - Groth16SecretKey
  - **`AuthorityDecryptionKey`**
    - Groth16DecryptionKey
  - …

The abstraction in more detail (2/2)

- Data formats defined by abstraction
  - **Claim Types** (`CTText`, `CTInt`, `CTAccumulatorMember`, or `CTEncryptableText`)
    - Describe attributes (in same order as Values)
    - `CTEncryptableText` enables special encoding required for decryption
  - **Data values** (`DVText` or `DVInt`)
    - Represent values in credential in some defined order
  - **ProofRequest** (a.k.a Presentation Request, etc.); really `Map CredentialLabel CredentialReq`
    - Defines what disclosures and properties Holder and Verifier agree on.
  - **SharedParams**; really Map SharedParamKey, SharedParamValue
    - `ProofRequest` refers to parameter values symbolically, `SharedParams` provides values
    - Makes `ProofRequest` reusable with different parameters and more readable
  - **DecryptRequest** (for Decryption)
    - Identifies credential and attribute to be decrypted, contains `AuthoritySecretData` and `AuthorityDecryptionKey`
  - **DecryptResponse** (for verifying Decryption)
    - Decrypted value, proof that it's correct

API functionality, expressed in Haskell-like syntax (1/2)

- Methods to create (abstract versions of): `SignerData`, `AuthorityData`, `AccumulatorData`, `RangeProofProvingKey`, `MembershipProvingKey`, `AccumulatorElement`
- ```
  type Sign =
       Natural   -- RNG seed
    -> [ DataValue ]
    -> SignerData
    -> Signature
  ```

- ```
  type AccumulatorAddRemove =
       AccumulatorData
    -> Accumulator
    -> Map HolderID AccumulatorElement
    -> [ AccumulatorElement ]
    -> AccumulatorAddRemoveResponse
  ```
  (includes accumulator witnesses, new accumulator value, accumulator update information)

- ```
  type UpdateAccumulatorWitness =
       AccumulatorMembershipWitness
    -> AccumulatorElement
    -> AccumWitnessUpdateInfo
    -> AccumulatorMembershipWitness
  ```

- ```
  type CreateProof =
       Map CredentialLabel CredentialReqs
    -> Map SharedParamKey SharedParamValue
    -> Map CredentialLabel
         SignatureAndRelatedData
    -> Maybe Nonce
    -> WarningsAndDataForVerifier
  ```
  (includes disclosed values)

For concrete examples, we assume a simple, artificial credential format

- We think of a “Credential Format Designer” role. CFD decides on format, rules, policy, etc. for what can be in a credential. Represents role of W3C, AnonCreds, etc.
- CFD defines how to map their credential format to the abstraction
- To enable experimentation and demonstration, we played this role by defining a simple Credential format (similar to JWT, but requires metadata that is signed as part of credential)

Example driver's license credential in our simple format

```
{ "metaDataForSimpleFormat" : {
      "purpose" : "DriverLicense",
      "version" : "1.0"
  },
  "sdAttrsForSimpleFormat": {
      "ssn":                        "123-12-1234",
      "eyeColor":                   "Brown",
      "daysBornAfterJan_1_1900":    37852,
      "height":                     180,
      "idForRev":                   "abcdef0123456789abcdef0123456789"
  }
}
```

"Credential Format Designer" requires a metadata attribute represented as a JSON object, and a flat list of named values (String or Int).

Values for example driver license credential


```
{"values":
  [ {"contents": "CredentialMetadata (fromList [(\"purpose\",DVText
\"DriverLicense\"),(\"version\",DVText \"1.0\")])",  "tag": "DVText"}
  , {"contents": 37852,                                "tag": "DVInt"}
  , {"contents": "Brown",                              "tag": "DVText"}
  , {"contents": 180,                                  "tag": "DVInt"}
  , {"contents": "abcdef0123456789abcdef0123456789", "tag": "DVText"}
  , {"contents": "123-12-1234",                        "tag": "DVText"}
  ]
}
```

Values for example driver license credential

```
{"values":
  [ {"contents": "CredentialMetadata (fromList [(\"purpose\",DVText
\"DriverLicense\"),(\"version\",DVText \"1.0\")])",  "tag": "DVText"}
  , {"contents": 37852,                                "tag": "DVInt"}
  , {"contents": "Brown",                              "tag": "DVText"}
  , {"contents": 180,                                  "tag": "DVInt"}
  , {"contents": "abcdef0123456789abcdef0123456789", "tag": "DVText"}
  , {"contents": "123-12-1234",                        "tag": "DVText"}
  ]
}
```

Note: The simple credential format we defined for experimentation requires metadata, encoded into a `DVText` as the first value, i.e., the attribute with index 0 in the list. The structure, contents, and encoding of this value are determined by the "Credential Format Definer", independent of the abstraction.

Claim Types for example driver license credential

```
{”claimTypes":
  [ "CTText"
  , "CTInt"
  , "CTText"
  , "CTInt"
  , "CTAccumulatorMember"
  , "CTEncryptableText"
  ]
}
```

Claim Types for example driver license credential

```
{”claimTypes":
   [ "CTText"
   , "CTInt"
   , "CTText"
   , "CTInt"
   , "CTAccumulatorMember"
   , "CTEncryptableText"
   ]
}
```

Note:

- `CTEncryptableText` indicates to Issuer to sign this attribute (Social Security Number in our example) for verifiable encryption; enables disclosure warnings, ”reversible encoding” that enables decryption without involvement of Prover, Verifier or Issuer.
- `CTAccumulatorMember` indicates that the value represents an element to be included in an accumulator; enables disclosure warnings

Proof Request for driver license and subscription use case (1/2)

```
{"driverLicense”:
  { ”signerPublicData”: "dlIssuer"
  , "disclosed"       : [0]
  , "inAccum"         : [[4, "dlCurrentAccum"]]
  , "notInAccum"      : []
  , "inRange"         : [[1, ["minBDdays", "maxBDdays", ”rangeProvingKey"]]]
  , "encryptedFor”    : [[5, "commonAuthorityPK"]]
  , "equalTo"         : [[5, ["subscriptionCred", 2]]]
  }
, …
```

Proof Request for driver license and subscription use case (1/2)

```
{"driverLicense”:
  { "signerPublicData": "dlIssuer"
  , "disclosed"       : [0]
  , "inAccum"         : [[4, "dlCurrentAccum"]]
  , "notInAccum"      : []
  , "inRange"         : [[1, ["minBDdays", "maxBDdays", ”rangeProvingKey"]]]
  , "encryptedFor”    : [[5, "commonAuthorityPK"]]
  , "equalTo"         : [[5, ["subscriptionCred", 2]]]
  }
, …
```

Note:

- These are indices into the list of Values; metadata is 0, Social Security Number is 5.
- Internal debate about whether to allow/require readable labels. Not strictly required, but small burden to provide and useful for understanding and debugging

Proof Request for example driver license and subscription use case (2/2)

```
…
,"subscriptionCred":
  { "signerPublicData”  : "subIssuer"
  , "disclosed"         : [0, 4]
  , "inAccum"           : [[1, "subCurrentAccum"]]
  , "notInAccum"        : []
  , "inRange"           : [[3, ["minValiddays", "maxValiddays", "provingKey"]]]
  , "encryptedFor"      : [[2, "commonAuthorityPK"]]
  , "equalTo"           : []
  }
}
```

Shared Params for example driver license and subscription use case

```
{ "dlIssuer"          : "eyJzcHNkU2lnUGFyYW1zIjoie1wiZzFcIjp...zUsNTBdIn0="
, "authorityPubData": "more opaque stuff"
, "maxBDdays"         : 999999999999
, "minBDdays"         : 37696
, …
}
```

Example of “opaque” data: different implementations may use different underlying cryptography, types, etc.

Separate minimal library requirements from higher-level functionality

AC/DC

Hyperledger ANONCREDS

W3C

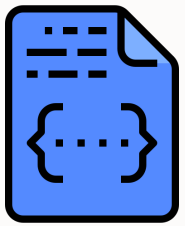
Platform API

Crypto Interface

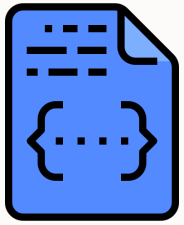
"General"

"Specific"

Hyperledger Ursa



Hyperledger AnonCreds

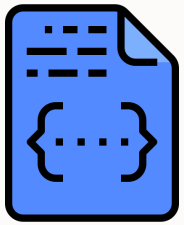


Hyperledger AnonCreds 2.0

...

Post-quantum?

DockNetwork crypto



See last slide for icon attributions (3,4)

Some benefits of separate CryptoInterface and PlatformAPI

- Individual “crypto implementers” (developers targeting our abstraction to a new underlying library) do not have to:
  - Deal with (or even be aware of) particular credential formats
  - Validate requirements
  - Issue warnings for potentially-unintended requirements
  - Lookup and and validate SharedParams
- What they do have to do
  - Define mapping for various data types (e.g., issuer keys, signatures, proofs, accumulators, …) to/from `OpaqueMaterial`, which is key to enabling abstraction
  - Provide library-specific implementations for, e.g.,
    - `CreateIssuerData, CreateAccumulatorData, CreateAuthorityData, Sign, AccumulatorAddRemove, UpdateAccumulatorWitness, CreateMembershipProvingKey, CreateRangeProofProvingKey`
  - Provide library-specific implementations for library-agnostic “proof instructions”, which are derived from proof requirements by the “General” transformation

Example of separation between “General” Platform API functions and “Specific” counterpart in Crypto Interface

Platform API

```
type CreateProof =
    Map CredentialLabel
        CredentialReqs
 -> Map SharedParamKey
        SharedParamValue
 -> Map CredentialLabel
        SignatureAndRelatedData
 -> Maybe Nonce
 -> WarningsAndDataForVerifier
```

Crypto Interface

```
type SpecificProver =
    [ProofInstructionResolved]
 -> EqualityReqs
 -> Map CredentialLabel
        SignatureAndRelatedData
 -> Nonce
 -> WarningsAndProof
```

Examples of “resolved” proof instructions

- Common to all Proof Instructions
  - `CredentialLabel`
  - `CredAttrIndex`
  - `RelatedIndex`
    - Example, for EncryptedFor proof instruction, index of Credential ProofInstruction specifying Proof of Knowledge of Signature covering the to-be-encrypted attribute.

- Credential
  - `SignerPublicData` (in opaque/abstract form)
  - Two-level map of revealed attributes
- InRange
  - Minimum and maximum values
  - `RangeProofProvingKey`
- InAccumulator
  - `AccumulatorPublicData`
  - `MembershipProvingKey`
  - `AccumulatorValue`
  - `BatchSequenceNum`
- EncryptedFor
  - `SharedParamKey`*
  - `AuthorityPublicData`

* redundant, convenient for distinguishing of multiple authorities

API functionality, expressed in Haskell-like syntax (2/2)

- ```
  type VerifyProof =
      Map CredentialLabel CredentialReqs
    -> Map SharedParamKey SharedParamValue
    -> DataForVerifier
    -> Map CredentialLabel
         (Map CredAttrIndex
            (Map SharedParamKey
                 DecryptRequest))
    -> Maybe Nonce
    -> WarningsAndDecryptResponses
  ```

  (includes 3-level map of `DecryptResponse`(s), which include decrypted values)

- ```
  type VerifyDecryption =
      Map CredentialLabel CredentialReqs
    -> Map SharedParamKey SharedParamValue
    -> Proof
    -> Map SharedParamKey
         AuthorityDecryptionKey
    -> Map CredentialLabel
         (Map CredAttrIndex
              (Map SharedParamKey
                   DecryptResponse))
    -> Maybe Nonce
    -> [ Warning ]
  ```

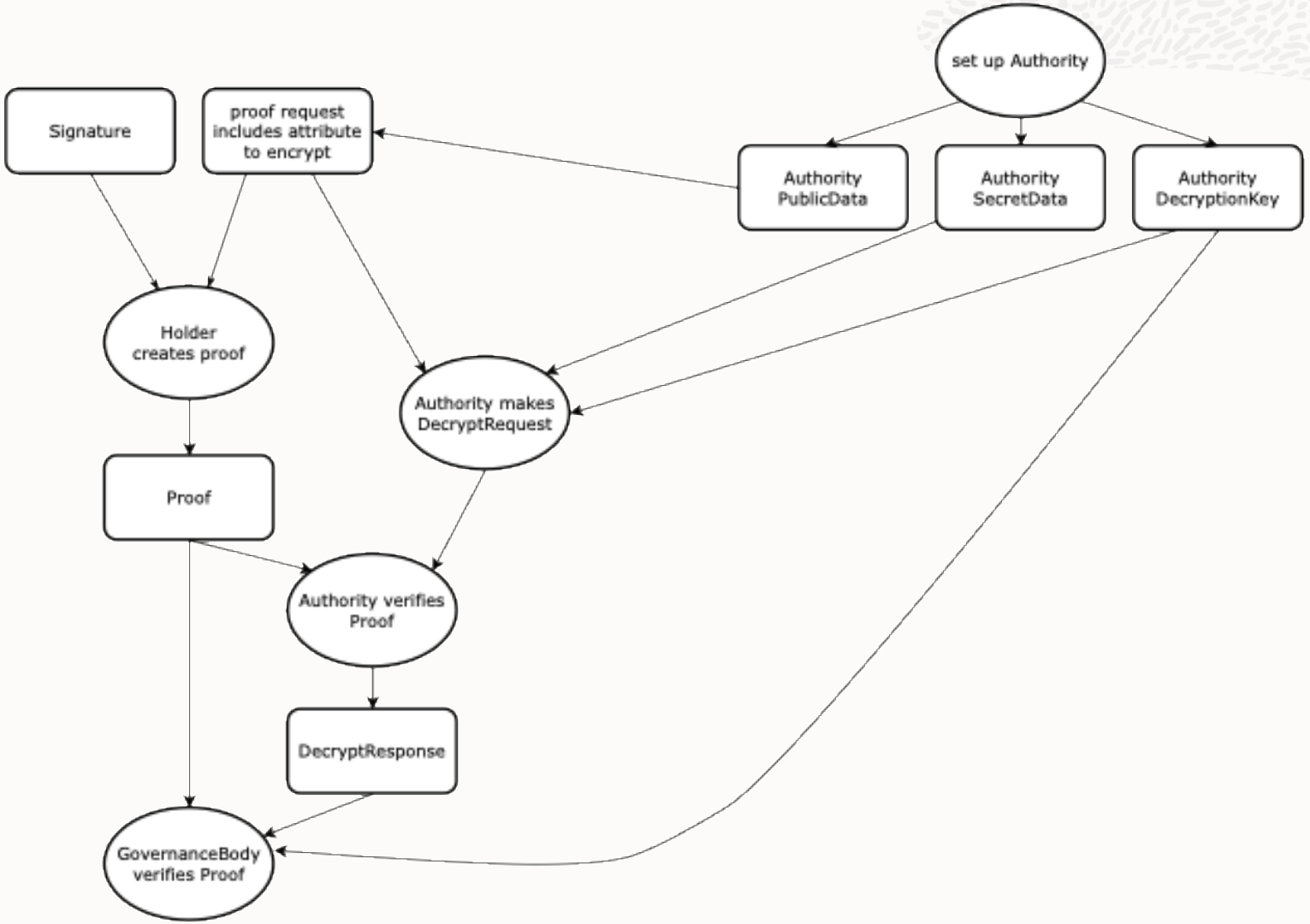
DecryptRequest(s) for example driver license and subscription use case

```
{"driverLicense":
        {5:
                {"Police": {"authSecret": "some opaque stuff"
                            ,"authDecryptionKey": "more opaque stuff"
                            }
                ,"OtherAuthority":
                            {"authSecret": "more opaque stuff"
                            ,"authDecryptionKey": "more opaque stuff"
                            }
                }
        }
,
…
}
```

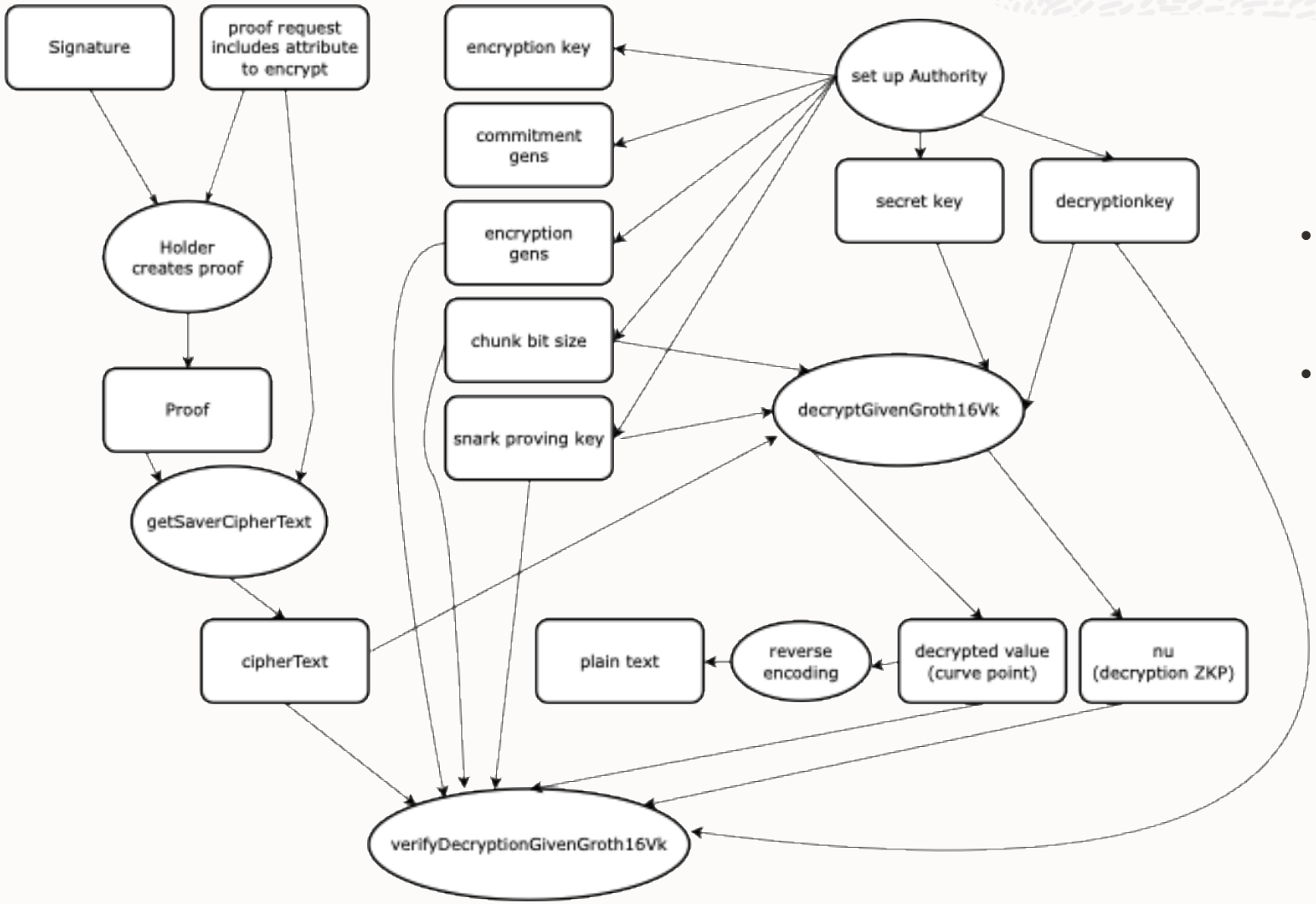
DecryptResponse(s) for example driver license and subscription use case

```
{"driverLicense":
        {5:
                {"Police": {”value": "123-12-1234"
                            ,"decryptionProof": "more opaque stuff"
                            }
                ,"OtherAuthority":
                            {"value": "123-12-1234"
                            ,"decryptionProof": "more opaque stuff"
                            }
                }
        }
,
…
}
```

Abstract view of Decryption and Decryption Verification



DNC-specific view of Decryption and Decryption Verification



- Did the Governance Body remember to also verify the proof?
- We remember for them, independent of underlying crypto library!

Agenda

1. “Elevator pitch” and research overview
2. Background and example use case
3. Motivation for and overview of our abstraction
4. Status: implementation and testing framework
5. Wrapup

Implementation status summary

- Internal prototype (Haskell)
  - Implemented `PlatformAPI` “General” pieces, enabling use with any crypto library that implements `CryptoAPI`
  - Implemented “Specific” pieces (using underlying Rust libraries via FFI utilities) for:
    - DockNetwork crypto (DNC)
    - AnonCreds v2 cryptography library (AC2C, decryption and decryption verification not yet supported)
- Public translation (Rust)
  - Translated “General” pieces to Rust
  - Translated “Specific” pieces for AC2C
  - TODO: Translation of “Specific” pieces for DNC
- Implemented previously, but not up-to-date with latest API:
  - Containerized http server serving API via auto-generated Swagger/API, facilitates easy experimentation from any language
  - Small test applications in Java, HTMX, TypeScript

Test framework

- Express test cases and expected outcomes concisely in JSON
- Test against any cryptography library that implements `CryptoInterface`
- Test common crypto-library-independent requirements, e.g.,
  - Number and value of revealed and decrypted attributes are same as requested
  - Decryption verification succeeds for correct value, fails for “perturbed” value
  - Out-of-date accumulator witnesses do not verify, updated ones do
  - Range proofs verify for boundary values, do not for out-of-range values
  - Equality proofs succeed for equal values, fail for different
  - Same value is provided for same attribute decrypted for multiple authorities
  - …

Test framework overview

- `TestState` represents “state of the world”
  - All data for all roles (Issuers, Holders, Authorities)
    - Single, unnamed verifier
- `TestSteps` modify and/or check properties of current `TestState`, e.g.,
  - Create setup data (e.g., `IssuerData`, `AuthorityData`, …), add requirements for Presentation by given Holder (PoK of sig, revealed attributes, equality, in range, in accumulator, encrypt for given Authority, decrypt by Authority, etc.)
  - Create and verify proof for given Holder, based on previously stated requirements
- Test steps include only human-friendly content, framework creates and manages Issuer/Authority/Accumulator setup params and keys, etc.

Example test scenario

- `CreateIssuer` (given label and schema)
- `CreateAccumulators` for Issuer (given its label)
- `SignCredential` (given labels for Issuer and Holder, and DataValues)
- `Reveal` some attributes (given Issuer and Holder labels, list of attribute indexes)
- `AccumulatorAddRemove` (given Issuer and Holder labels, attribute index, add value to accumulator)
- `InAccum` (given Issuer and Holder labels, attribute index, sequence number 1)
- `CreateAndVerifyProof`, expect both to succeed without warnings

Example test scenario – JSON (same test passes in Haskell and Rust)

```
{"descr": "example_single_issuer_and_credential_in_accum_no_update",
 "provenance": "DO NOT MODIFY MANUALLY: this test was generated programatically; … <more info to
enable us to identify original Haskell code>",
 "testseq": [{"contents": ["DMV",
                           ["CTText", "CTInt", "CTText", "CTInt", "CTAccumulatorMember"]],
              "tag": "CreateIssuer"},
             {"contents": "DMV", "tag": "CreateAccumulators"},
             {"contents": ["DMV", "Holder1",
                           [{"contents": "some metadata", "tag": "DVText"},
                            {"contents": 37852, "tag": "DVInt"},
                            {"contents": "Brown", "tag": "DVText"},
                            {"contents": 180, "tag": "DVInt"},
                            {"contents": "abcdef0123456789abcdef0123456789",
                             "tag": "DVText"}]],
              "tag": "SignCredential"},
             {"contents": ["DMV", 4,
                           {"Holder1": {"contents": "abcdef0123456789abcdef0123456789",
                                        "tag": "DVText"}},
                           []],
              "tag": "AccumulatorAddRemove"},
             {"contents": ["Holder1", "DMV", [0, 3]], "tag": "Reveal"},
             {"contents": ["Holder1", "DMV", 4, 1], "tag": "InAccum"},
             {"contents": ["Holder1", "BothSucceedNoWarnings"],
              "tag": "CreateAndVerifyProof"}]}
```

Example test scenario – Haskell (used to create JSON on previous slide)

```
example_single_issuer_and_credential_in_accum_no_update =
  TF.TestSequenceWithMetadata
    provenance
    "example_single_issuer_and_credential_in_accum_no_update"
    Nothing
    [ CreateIssuer "DMV"   [ CTText, CTInt, CTText, CTInt, CTAccumulatorMember ]
    , CreateAccumulators "DMV"
    , SignCredential "DMV" "Holder1" [ DVText "some metadata"
                                     , DVInt 37852
                                     , DVText "Brown"
                                     , DVInt 180
                                     , DVText "abcdef0123456789abcdef0123456789"]
    , AccumulatorAddRemove "DMV" 4
        (fromList [("Holder1", DVText "abcdef0123456789abcdef0123456789")]) []
    , Reveal "Holder1" "DMV" [0, 3]
    , InAccum "Holder1" "DMV" 4 1
    , CreateAndVerifyProof "Holder1" BothSucceedNoWarnings
    ]
```

TestSteps and their parameters

```
CreateIssuer                IssuerLabel [ ClaimType ]
SignCredential              IssuerLabel HolderLabel [ DataValue ]
CreateAccumulators          IssuerLabel
AccumulatorAddRemove        IssuerLabel CredAttrIndex (Map HolderLabel DataValue)
                                                      [ DataValue ]
Reveal                      HolderLabel IssuerLabel [ CredAttrIndex ]
InRange                     HolderLabel IssuerLabel CredAttrIndex Word64 Word64
InAccum                     HolderLabel IssuerLabel CredAttrIndex AccumulatorBatchSeqNo
Equality                    HolderLabel IssuerLabel CredAttrIndex
                                                    [(IssuerLabel, CredAttrIndex)]
CreateAndVerifyProof        HolderLabel CreateVerifyExpectation
UpdateAccumulatorWitness    HolderLabel IssuerLabel CredAttrIndex AccumulatorBatchSeqNo
CreateAuthority             AuthorityLabel
EncryptFor                  HolderLabel IssuerLabel CredAttrIndex AuthorityLabel
Decrypt                     HolderLabel IssuerLabel CredAttrIndex AuthorityLabel
VerifyDecryption            HolderLabel
```

Notes on `TestStep` effects

- Not all possible scenarios can be modeled, e.g.,
  - At most one credential can be signed for the same Issuer and Holder (enables using `AuthorityLabel`s as `CredentialLabel`s)
- To simplify tests, some `TestStep`s model multiple (assumed) real-world events, e.g.:
  - `Sign` step records the signature produced in the Holder's state, modeling the Issuer sending it, the Holder receiving it and storing it
  - `AccumulatorAddRemove` step receives `HolderLabel`(s) and `DataValue`(s), and records initial `AccumulatorMembershipWitness` for added values in respective Holders' state in `TestState`.
    - In reality, a "revocation manager" (separate from the Issuer requesting the additions) does not need to know Holder's identity or raw `DataValue` from which accumulator element is derived, providng these to `AccumulatorMembershipWitness` step simplifies tests
    - Witnesses could be sent back to Issuer, who forwards to relevant Holder(s), who store them for later use.
  - `UpdateAccumulatorWitness` step receives a target "batch sequence number", determines what updates are needed, extracts them from `TestState` and applies them in order
    - In reality, these have to be published by the Revocation Manager and received (possibly indirectly) by the Holder and applied individually

Expectations and overrides

- `CreateAndVerifyProof TestStep` takes a `CreateVerifyExpectation` (expected outcome of test). Test outcomes (so far):
  - `BothSucceedNoWarnings`
  - `CreateProofFails`
  - `VerifyProofFails` (implies proof creation succeeds)
  - `CreateOrVerifyFails` (in some cases, some implementations refuse to create proofs, while others fail verification)
- Including `expected_to_fail` in a test name requires it to fail
- We can (per-cryptography-library) override (i.e., don't run) specific tests because:
  - `SLOW`
  - `Fail`: known to fail, e.g., because specific library has a known bug or limitation that causes tests to crash
  - `Skip`: don't want to run it for some other reason

Feature and test coverage status

	Haskell (internal)		Rust (open source)		JSON test coverage
	DNC	AC2C	DNC	AC2C	
PoK of Signature	✅	✅	☐	✅	✅
Selective disclosure	✅	✅	☐	✅	✅
Equality	✅	✅	☐	✅	✅
Range proof	✅	✅	☐	✅	✅
Set membership, including updates	✅	✅	☐	✅	✅
Verifiable Encryption	✅	✅	☐	✅	✅
Decryption	✅	NYS	☐	NYS	✅
Verify Decryption	✅	NYS	☐	NYS	✅

*NYS = Not Yet Supported (by underlying cryptography library)

Status, caveats, code availability

- Test expectations limited, e.g., can’t say anything about warnings except they don’t exist
- Overriding system not fully translated from internal Haskell, not super happy with internal version, different test mechanics in Haskell and Rust complicate things
- Several developers, different levels of experience with both languages, familiarity with system, “taste”, etc. Some code more Rusty, some more “Haskell translated to Rust”, etc. Code is far from “polished”!
- But we are quite happy with the abstraction, success implementing it over multiple cryptography libraries, ability to run and add tests as JSON files, etc.
- Not letting “perfect” be the enemy of “good”, we have pushed a branch to our public fork:
  - https://github.com/mark-moir/anoncreds-v2-rs/pull/1

Agenda

1. “Elevator pitch” and research overview
2. Background and example use case
3. Motivation for and overview of our abstraction
4. Status: implementation and testing framework
5. Wrapup

So far we have…

- Defined an initial abstraction, embodied by Swagger/OpenAPI
- Used it to develop a Car Share Service use case demo involving Driver's License and Monthly Subscription credentials, keeps hirer anonymous, enables identification by Authority if needed
- Implemented the abstraction (in an internal Haskell prototype) over:
  - DockNetwork crypto (DNC, https://github.com/docknetwork/crypto)
  - AnonCreds 2.0's cryptography library (AC2C, https://github.com/hyperledger/anoncreds-v2-rs)
- Built a Docker container that serves (an early version of) the API via internal Haskell prototype
- Continued refining and extending the API
- Developed a framework to express tests via JSON, independent of underlying library, and run them
- Translated the "General" pieces of our prototype to Rust
- Translated the AC2C "Specific" pieces to Rust
- Translated our test framework to Rust
- Open sourced our preliminary Rust implementation for feedback and engagement

Ongoing and future work

- Short-to-medium term
  - Continuing improvement of Rust translation, taking into account feedback from community
  - Working towards contributing it to https://github.com/hyperledger/anoncreds-v2-rs
  - Translating DNC-based implementation of our abstraction to Rust
  - (Internal) engaging with a product group towards enabling use of our abstraction to express and implement use cases via our abstraction
  - Update containerized server to facilitate experimentation (in Haskell for internal use and/or Rust for broader availability)
- Longer term
  - Extend abstraction to support additional features. Example: AnonCreds 2.0 aims to support ALLOSAUR, current accumulator abstraction probably won't accommodate it
  - Continue engagement with more product groups, facilitate experimentation with more use cases, credential formats
  - Investigate implementing abstraction using combinations of different underlying cryptography libraries

Summary

- We are:
  - Learning about technologies, projects, and standards efforts around Verifiable Credentials and Zero Knowledge Proofs
  - Demonstrating (internally) potential of these technologies
  - Developing an abstraction to decouple VCs from underlying cryptography
  - Implementing and testing our abstraction, instantiated with multiple cryptography libraries
  - Sharing our experience and work so far, seeking feedback, engagement
  - Contributing to `anoncreds-v2-rs`: a few bug fixes and generalisations already merged, recently pushed abstraction implementation, implementation over AC2C, test framework and tests to public fork for feedback towards contribution

Thank you

Mark Moir, Architect, Oracle Labs

mark.moir@oracle.com

www.linkedin.com/in/markmoir

Icon attributions

All icons by Freepik from flaticon.com, except:

1. Private message icons created by juicy_fish – Flaticon
2. Arrest icons created by shmai – Flaticon
3. Icon by ultimatearm
4. By Denmokin - Own work, Public Domain
5. Professional icons created by Uniconlabs - Flaticon